UNIGE - Robotics Engineering - AI4R2 - 2026

Assignment Report: Single Robot Pick-and-Deliver

Daneri Gregorio / n°5658523

Introduction

The assignment requires modelling a warehouse environment composed of multiple locations using the PDDL language. The scenario features a mobile robot whose goal is to collect packages from storage areas and deliver them to specified destinations. The robot can carry only one package at a time and must navigate between locations using predefined connections.

The project comprises two separate models:

- PDDL model: implements basic predicates and actions to represent the situation.
- PDDL+ model: in addition to the previous design introduces the concepts of missed deadlines and non-instantaneous transportation.

PDDL model

The domain provides simple navigation capabilities within a multi-structured environment, together with package manipulation through pick-up and drop-off actions.

Predicates

- **at** and **package-at** provide an explicit representation of both the robot and the package position.
- **free** and **holding** ensure that the robot can pick up only one package at a time and show which package is being currently carried. They may seem redundant. However, using only the first predicate would not be enough to tell which package is being carried, while the second one would not prevent the robot, already carrying a package, from picking up a different one.

- **connected** defines the warehouse structure by linking different locations, thus allowing the robot to transit between them. It is not symmetric by default, so, to connect areas in both ways, it must be made explicit in the problem definition.
- **goal-location** states the final destination for each package, while **delivered** states if the package has been dropped off at its destination, avoiding multiple dispatches for the same package.
  To correctly define a problem, each package must be associated with a location of the goal; otherwise, the planner will not consider those items.

Actions

- ***move*** allows the robot to navigate between locations connected to the current one. The action does not allow the robot to skip locations and is not symmetric by default.
- ***pick-up*** and ***drop-off*** explicitly implement package manipulation, allowing the robot to hold only one item at a time.

It is assumed that the robot can take or leave packages only when it stops at a location. However, since actions are instantaneous, it is not necessary to model this feature.

PDDL problems

The model was tested in two scenarios.

- Single package delivery: the robot has to deliver only one package, so the only constraint is that the robot can move only between connected locations.
- Multiple packages with sequential delivery: in addition to the movement limitation, the planner has to redefine its main goal, as the package needed to be delivered changes, thus requiring more flexibility in planning.

The PDDL domain structure allows plans to be generated for problems involving any number of packages. The number of packages does not affect the feasibility of the model itself, although larger problems naturally lead to longer plans.

Output Single package delivery

In this simple scenario, the result verifies the expectation: the robot moves through the various locations towards the closest package, once reached, it picks it up, and it delivers it to its destination.

(a) Planner output

```
Plan found:
0.00000: (move r l1 l2)
0.00100: (move r l2 l3)
0.00200: (move r l3 l4)
0.00300: (move r l4 l5)
0.00400: (pick-up r p l5)
0.00500: (move r l5 l4)
0.00600: (move r l4 l3)
0.00700: (move r l3 l2)
0.00800: (drop-off r p l2)
Metric: 0.008
Makespan: 0.008
States evaluated: undefined
Planner found 1 plan(s) in 3.545secs.
```

(b) Terminal output

Output Multiple packages delivery

(a) Planner output

```
Plan found:
0.00000: (pick-up r p1 l1)
0.00100: (move r l1 l2)
0.00200: (drop-off r p1 l2)
0.00300: (pick-up r p3 l2)
0.00400: (move r l2 l3)
0.00500: (move r l3 l4)
0.00600: (move r l4 l5)
0.00700: (drop-off r p3 l5)
0.00800: (move r l5 l4)
0.00900: (move r l4 l3)
0.01000: (pick-up r p2 l3)
0.01100: (move r l3 l4)
0.01200: (drop-off r p2 l4)
Metric: 0.012000000000000004
Makespan: 0.012000000000000004
States evaluated: undefined
Planner found 1 plan(s) in 3.511secs.
```

(b) Terminal output

As in the previous problem, the delivery process adheres to the most likely prospects: the robot sequentially visits the various locations, and either picks up or drops off packages in the same order it encounters them, without behaving unexpectedly. In problems without timing constraints, the robot gives priority to the closest item to pick up, if it is not holding any, so the sequence of its actions can be easily foreseen a priori.

PDDL+ model

This model implements the same features of the previous domain, but also represents the time spent during the robot motion and simulates missed deadlines in package distribution. This is achieved by employing PDDL+ constructs such as processes and events, and by splitting the robot movement into two steps. As a result, robot transportation is no longer instantaneous.

Predicates

Supplementary to those displayed in the previous design:

- **moving** and **transiting** are used to correctly carry out the movement action, ensuring that the movement is no longer instantaneous. In particular, the first predicate, when set to true, is used to trigger the process that simulates the passage of time, as well as preventing the robot from picking up or dropping off items during motion. Indeed, in a real scenario, it would be natural to guarantee that these operations are executed safely. The second one is used to link the two actions that make up the motion of the robot: more precisely, they ensure that the rooms used in the first step are also those used in the second one. Otherwise, the robot might be able to move to a room not directly connected to the initial one, thereby violating the rule of "*no teleportation*".
- **overdue** notifies the planner if a package has been delivered beyond its deadline. When defining a problem, it is possible to request that a package must NOT be **overdue**. In that case, the item must be delivered within its deadline; otherwise the plan will not be valid.

Functions

These fluents have been added to model time, spent during transportation, and deadlines:

- *elapsed-time* tracks the global time which is necessary to identify delays.
- *travel-time* represents the amount of time necessary to move from one location to another.
- *move-progress* keeps track of the robot's movement between two locations.
- *deadline* defines for each package the time limit within which it must be delivered, checked with respect to *elapsed-time*.If an item is delivered after its deadline, **overdue** is set to `TRUE`.

Processes

They allow the values of the fluents to change linearly with time:

- TIME-PASSAGE models the passage of global time, which is used to trigger delays when deadlines are not met.
- MOVING-PROCESS simulates the progression of the robot's movement between two sections by increasing *move-progress*.

Actions

Conceptually, they are the same as those of the previous domain, but there are some changes due to the introduction of time:

- ***pick-up*** and ***drop-off*** can be executed only when the robot is not moving.
- ***start-move*** and ***finish-move*** define the two steps of robot motion. The sequence of these actions is controlled by *move-progress*, which, after executing ***start-move***, is linearly increased with time by MOVING-PROCESS. Once the threshold *travel-time* is reached, ***finish-move*** executes, allowing the robot to pick-up/drop-off objects or move again.

Effect

- DEADLINE-MISSED sets a package as **overdue** if it has not been delivered within its deadline. Depending on the problem definition, this may cause it to be unsolvable. However, in some scenarios, it may be acceptable to let robots deliver items late.

PDDL+ problems

As for the previous model, two problem files have been defined to test the domain. In particular, the first one manages one package, while the latter two both with a separate deadline. To configure the scenarios it is necessary to initialise all the fluents before execution:

- The initial global time value (usually zero).
- The time required to travel between each pair of connected locations. The value must be defined for both directions, so that the robot can move back and forth.
- The deadline for each package defined in the problem. This value is defined with respect to the global time.

Obviously, the robot is initially halted, so *moving* must be omitted from the initialisation. Concerning the objectives, the robot has to deliver all packages within the corresponding time limit. However, it is also possible not to consider deadlines, thus simplifying the goal condition.
In any case, the problem requires that the objective must be reached in the shortest possible time, thanks to the applied metric.

Outputs

```
Found Plan:
0: -----waiting---- [1.0]
1.0: (start-move r l1 l2)
1.0: -----waiting---- [6.0]
6.0: (finish-move r l1 l2)
6.0: (pick-up r p1 l2)
6.0: (start-move r l2 l3)
6.0: -----waiting---- [11.0]
11.0: (finish-move r l2 l3)
11.0: (start-move r l3 l4)
11.0: -----waiting---- [16.0]
16.0: (finish-move r l3 l4)
16.0: (start-move r l4 l5)
16.0: -----waiting---- [21.0]
21.0: (finish-move r l4 l5)
21.0: (drop-off r p1 l5)
21.0: -----waiting---- [22.0]
22.0: (start-move r l5 l4)
22.0: -----waiting---- [28.0]
28.0: (finish-move r l5 l4)
28.0: (pick-up r p2 l4)
28.0: (start-move r l4 l3)
28.0: -----waiting---- [33.0]
33.0: (finish-move r l4 l3)
33.0: (start-move r l3 l2)
33.0: -----waiting---- [38.0]
38.0: (finish-move r l3 l2)
38.0: (start-move r l2 l1)
38.0: -----waiting---- [43.0]
43.0: (finish-move r l2 l1)
43.0: (drop-off r p2 l1)
```

(a) Two packages with deadlines respectively of 25 and 45 time units

```
Found Plan:
0: (start-move r l1 l2)
0: -----waiting---- [5.0]
5.0: (finish-move r l1 l2)
5.0: (pick-up r p l2)
5.0: (start-move r l2 l3)
5.0: -----waiting---- [10.0]
10.0: (finish-move r l2 l3)
10.0: (start-move r l3 l4)
10.0: -----waiting---- [15.0]
15.0: (finish-move r l3 l4)
15.0: (start-move r l4 l5)
15.0: -----waiting---- [20.0]
20.0: (finish-move r l4 l5)
20.0: (drop-off r p l5)
```

(b) One package with a deadline of 25 time units

In these problems, the behavior of the planner can change depending on how many packages are present and their deadlines. However, as in the problems of the previous model, without a custom scheduling algorithm for deliveries, the robot prioritise the item that encounters first. Obviously, this choice can lead to non-optimal plans and even prevent the planner from finding the solution, for instance: let us consider a scenario where the first package encountered has a farther deadline than the one of an item that could have been picked up next. In that case, if the amount of time required to deliver the first parcel, pick up the second one, and deliver it as well, exceeded its deadline, then, without the right policy to choose which item pick up first, the problem would be unsolvable.

Discussion

The current models present mainly two limitations:

- Scalability in multiple packages problems, which, when implemented, require the robot to repeat the navigation process several times, hence possibly generating conflicts between shipments, missed deadlines, and inefficient delivery order. In those cases, planning complexity grows rapidly, action sequences become much longer, and deadlines are harder to meet.
- The inability to perform non-sequential actions prevents the system from implementing a variety of additional features such as simultaneous deliveries, parallel motions and coordinated behavior, considering multiple robots. Indeed, in a real scenario, a warehouse usually allows a rather high degree of concurrency with actions such as charging while waiting, or multiple couriers working simultaneously.

Possible upgrades to overcome these issues:

- Multiple robots operating in parallel
- Carrying multiple packages simultaneously

However, in order to be in line with reality, these changes must be followed by additional modifications that further increase the complexity of the model. Some of them may be as follows:

- Modelling the maximum payload, as well as the space capacity of a robot, and the actions necessary to take in case these thresholds are reached. A possible solution could consist of additional robots taking care of the packages left unattended.
- Defining a virtual map for each robot to keep track of the destination of every package it has picked up. This strategy could also be paired with a route optimisation algorithm, for instance to cluster the delivery of nearby packages.
- Synchronizing robot movements to avoid collisions. As in the previous point, also in this case, each robot would need a virtual representation of the position of other workers. An alternative or complementary solution could be to define fixed paths for each robot, making it easier to prevent collisions. However, if not properly managed, this strategy could lead to deadlocks depending on how couriers roam the environment.
- Selecting and delivering packages based on the priorities assigned to them. For this implementation, it would be recommended to apply some real-time scheduling algorithm such as Rate Monotonic.

As can be seen, many complications arise from potential improvements; additional features always come at a cost. In particular, both implementing a system of concurrent deliveries with multiple robots and enhancing robot load capacity require addressing timing constraints, which are already present, although in a

simpler form, in the basic PDDL+ model. These limitations can be partially mitigated through optimisation algorithms capable of finding shorter and faster routes to the target locations.
A more robust solution would be to address directly the concurrency introduced either by multiple packages carried by a single robot or by multiple robots operating simultaneously. However, in the latter case, another issue emerges: shared resources. Multiple robots may need to deliver similar packages concurrently, requiring access to the same location. Therefore, it becomes necessary to define constraints on the maximum number of robots allowed within a location at the same time. Such situations would require the introduction of guarding mechanisms such as semaphores, mutexes, or condition variables.

Conclusion

The implemented warehouse model is far from providing all the features necessary to represent a real-world scenario; many improvements would be required to simulate a realistic working environment. However, efficiency and resource consistency become a major concern when adding those functionalities, leading to a significant increase in complexity.